

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Design, brute-force validation, and a controlled comparison with Q3C and pgSphere

J. Salgado¹  and M. Delli Veneri¹ 

SKA Observatory, Jodrell Bank, Lower Withington, Macclesfield SK11 9FT, United Kingdom
jesus.salgado@skao.int, michele.delliveneri@skao.int

Received ...; accepted ...

ABSTRACT

Context. Astronomical archives answer positional queries from relational databases, and in the PostgreSQL world two extensions dominate: Q3C, which keys each source by an integer drawn from a quad-tree on a cube and stores it in a B-tree, and pgSphere, which stores spherical geometry in a GiST tree. The first is compact and fast for points; the second is general and indexes regions as well.

Aims. We ask whether the two can be combined: a single index that keeps the integer key and the B-tree, yet answers cones, polygons, cross-matches and stored footprints in archives’ own geometry language, establishing where it helps, where it does not, and what its cost model depends on.

Methods. We present *skycell*, a PostgreSQL extension that keys positions by their order-29 HEALPix cell in the nested scheme and turns a query region into index ranges. How finely to cut is decided by a cost model – the price of an extra index range against the rows it removes – with the local source density read from the histogram *ANALYZE* already keeps for the key column. Cells are classified conservatively against the region, and a planner support function rewrites the predicate into B-tree range conditions plus an exact test. Coverings are validated by brute force against millions of catalogue points, including adversarial cones and polygons. Performance is measured against Q3C 2.0.5 and pgSphere 1.5.2 on two 10-million-row corpora – one synthetic, one resampled from the real Gaia DR3 density field – in trials where all three methods answer the same query back to back in a randomized order, analysed paired, warm and with caches dropped, and repeated on a 50-million-row catalogue where pgSphere’s index no longer fits in memory.

Results. Warm, *skycell* is faster than pgSphere by 6–11% for cone radii of 1′–1′ and by 31–37% at 3°; between 6′ and 1° it is indistinguishable on the designed corpus, and on the Gaia corpus 3% slower at 6′ but 8–15% faster at 30′–1°; cold it is faster at every radius on both corpora (ratios 0.39–0.95). It keeps an index of 214 MB against pgSphere’s 685 MB – 22.5 bytes per row against 72, which at Gaia DR3’s 1.8×10^9 sources is 38 GiB against the 160 GiB the pgSphere-plus-Q3C pair needs to cover both cone searches and cross-matching, builds in 1.4 s against 42 s, answers convex polygons 2.7–4.4 times faster, matches *q3c_join* on cross-matching while being 2.5–3.3 times faster than pgSphere at it, and its row estimates are wrong by a factor 1.2 rather than 2.5–3.0. The cost model’s one parameter is derived from the relation’s own statistics rather than configured, its chosen cell order costs about 3% against the best available one, and its density estimate is accurate to about 25% on ordinary sky. Raising the catalogue to 50 million rows widens the advantage for regions of 30′ and larger (at 1°, 0.85 to 0.60 warm, 0.53 to 0.31 cold) while cones of an arcminute and below do not move. An ObsCore-shaped relation measured at three sizes puts the crossover with pgSphere between 0.5 and 2 million rows for degree-scale cones, and beyond 10 million for small cones on field-clustered data.

Conclusions. A cost-based covering over an equal-area integer key is a sound basis for a sky index, and the right question is not whether it is faster but for which queries. Its cost is paid per query and its benefit per row scanned, so it wins where enough rows are scanned to amortise a covering – degree-scale regions, polygons, cross-matching, cold caches – and that boundary moves up in radius as the catalogue grows denser. On a Gaia-sized catalogue a small cone search is not the workload this index improves.

Key words. Astronomical databases: miscellaneous – Catalogs – Virtual observatory tools – Methods: data analysis

1 Introduction

Almost every question asked of an astronomical archive is positional: what is within this cone, which observations cover this field, which of my sources match that catalogue. In the Virtual Observatory these questions arrive as ADQL (Ortiz et al. 2008) through the Table Access Protocol (Dowler et al. 2019), and the service has to turn them into something a relational database can answer with an index.

Two PostgreSQL extensions carry most of this

traffic. Q3C (Koposov & Bartunov 2006) projects the sphere onto a cube, subdivides each face with a quad-tree, and numbers the cells along a space-filling curve, so each source is one 64-bit integer in a B-tree and the table can be sorted along the same curve. pgSphere (Chilingarian et al. 2004) takes the other route: it adds spherical types – points, circles, polygons, boxes – and indexes them with GiST over bounding volumes, so regions can be stored and queried as data, not only as query arguments.

The scale this has to work at is set by archives al-

ready in operation. The ESA Gaia Archive (Salgado et al. 2017, 2020) serves order 10^9 sources through an extended TAP interface, and its published description puts cross-match – matching a user’s uploaded positions against the catalogue – among its server-side capabilities, so Sect. 6.3 treats it as a first-class result rather than an afterthought.

Neither idea is new to astronomy: the Hierarchical Triangular Mesh (Kunszt et al. 2001) already keyed positions by a subdivided polyhedron, and the zones algorithm (Gray et al. 2006) showed how far a purely relational arrangement can be pushed for cross-matching. What has changed is the size of the catalogues and the fact that most queries now arrive through a generated layer (e.g. Demleitner et al. 2014) rather than being written by hand.

The two designs trade off against each other. A linear key in a B-tree is compact, cheap to probe and orders the heap, but a region has to be turned into a set of key ranges before the index can be used, and that approximation’s quality depends on how finely the region is cut. A tree of bounding volumes has no such step and handles extended objects naturally, but its keys are larger, its nodes overlap, and it imposes no useful order on the table.

This paper presents *skycell*, which keeps the integer key and asks a cost question at the point where the classical approach fixes a constant: how finely should a region be cut? The answer depends on two quantities the database already knows – how many sources sit in that part of the sky, and how much an extra index range costs – and can be computed rather than configured.

Because the claims here are quantitative, the measurements are built to survive scrutiny rather than to flatter the method: every query is answered by all three methods back to back in a randomized order, repeated and analysed paired; everything is measured both warm and with the page cache dropped; and everything is repeated on a second corpus whose density structure comes from the real Gaia DR3 sky rather than from the authors. Section 2 develops the method, Sect. 3 its planner integration, Sect. 4 what is proven versus tested, Sect. 5 the benchmark protocol, Sect. 6 the results, and Sect. 7 a TAP service and the measured crossover; Sect. 8 states plainly what these measurements do not establish.

2 Method

2.1 An equal-area integer key

A position is keyed by the index of the HEALPix cell containing it at order 29 in the nested scheme (Górski et al. 2005) ($12 \cdot 4^{29} \approx 3.5 \times 10^{18}$ cells of about 0.4 mas), computed by the standard *ang2pix_nest* arithmetic and stored as a signed 64-bit integer – the plain nested index at a fixed order, not a multi-order

key, so a key is directly comparable with another and a B-tree orders them along the HEALPix curve. Three properties follow, and only the first is shared with Q3C’s cube quad-tree.

Cells are *nested*: the order-29 descendants of a cell p at order k are exactly the contiguous interval

$$[p \cdot 4^{29-k}, (p+1) \cdot 4^{29-k} - 1], \quad (1)$$

so a region covered by cells of any orders becomes a union of integer intervals and a B-tree can answer it with range scans. The NUNIQ encoding of the MOC standard (Fernique et al. 2014), used here for stored regions (Sect. 2.2), packs the order into the key as $4 \cdot 4^k + p$; the two are related by Eq. (1), and *skycell* converts between them rather than storing two representations.

Cells are *equal area*, so a count of keys in an interval divided by its length in cells is proportional to a source density per unit solid angle: the histogram *ANALYZE* already keeps for the key column is therefore a crowding map (Sect. 2.2) with no extra statistics machinery. And cells are *standard* – the same numbering underlies Gaia’s *source_id* (Gaia Collaboration 2016) and IVOA Multi-Order Coverage maps (Fernique et al. 2014) – so keys, coverings and stored footprints are interoperable rather than private to one extension.

2.2 Coverings decided by cost

A query region is covered by cells whose union contains it; the covering becomes range conditions on the key, and rows inside the covering but outside the region are removed by an exact test. Cutting finer means fewer such false positives and more ranges. With cells of size s covering a region of characteristic radius r and a local source density ρ ,

$$C(s) = n_{\text{ranges}}(s) c_{\text{range}} + \rho (A_{\text{cover}}(s) - A_{\text{region}}). \quad (2)$$

Both terms are in rows: the second is the expected number of false positives, and c_{range} converts one additional index range into the number of false-positive rows it is worth avoiding – a property of the installation, not of the sky (the ratio between the cost of opening one more B-tree range scan and of fetching and discarding one heap row; Sect. 6.4 measures it).

A covering follows the region’s boundary, so for $s \ll r$ the number of ranges grows as r/s – the curve enters and leaves the region once per boundary crossing, and the number of such clusters is the classical measure of a space-filling curve’s quality (Moon et al. 2001) – while the wasted area grows as the perimeter times s . Writing $A_{\text{cover}} \simeq \pi(r + \beta s)^2$ and $n_{\text{ranges}} \simeq \gamma r/s$ and minimising Eq. (2) gives

$$s_{\star} = \sqrt{\frac{\gamma c_{\text{range}}}{2\pi\beta\rho}}, \quad (3)$$

which does not depend on the size of the region: the density of the sky and the price of a range decide how finely it is worth cutting. For $s \gtrsim r$ the covering is a handful of cells and the balance differs, so rather than solve the branches the implementation evaluates Eq. (2) at all 30 orders – a few floating-point operations each – and takes the cheapest, which also lets a cap on the number of ranges and a guard on cell size simply remove candidates. The constants $\beta = 0.7$ and $\gamma = 2$ are geometric; Sect. 6.4 shows the choice of order is flat enough near its optimum that they do not need to be exact.

Figure 1 shows the two extremes on the same cone radius: at high Galactic latitude the covering is a single range of large cells covering 5.6 times the cone’s area, because there is almost nothing to exclude; in the bulge the same cone is covered by 23 cells in 8 ranges covering 1.3 times its area, because there the rows removed are worth the extra ranges.

Two guards keep the model safe against its own inputs: a partly covered cell is never allowed to exceed a multiple (64 by default) of the region’s own area, bounding the damage from a dense cluster the histogram underestimates; and gaps between ranges are merged when the rows they contain cost less than the range they save.

The density estimate. PostgreSQL’s `ANALYZE` stores, for the indexed expression, an equi-depth histogram: a sorted list of $B+1$ bounds $h_0 < h_1 < \dots < h_B$ in key space such that each bucket $[h_i, h_{i+1})$ holds about N/B of the table’s N rows. Because the key is an equal-area cell index, the rows in a key interval $[a, b]$ are estimated by linear interpolation within the buckets it spans,

$$\hat{n}(a, b) = \frac{N}{B} \sum_i \frac{|[a, b] \cap [h_i, h_{i+1})|}{h_{i+1} - h_i}, \quad (4)$$

and the corresponding density in rows per steradian is

$$\hat{\rho}(a, b) = \frac{\hat{n}(a, b)}{(b - a + 1) A_{29}}. \quad (5)$$

The estimate used by Eq. (3) is $\hat{\rho}$ evaluated over the order- m cell containing the region’s centre, with m chosen so that $s_m \approx 4r$: large enough to contain enough rows to be a density, small enough to be local.

Three caveats. The quantity is a density per unit solid angle, not a selectivity – PostgreSQL computes the planner’s own selectivity for the rewritten range conditions separately, from the same histogram. Equation (4) assumes rows are spread uniformly inside a bucket, which fails for a cluster smaller than one – hence the area guard of Sect. 2.2. And the estimate is only as local as a bucket: with $B = 1000$ and $N = 10^7$ a bucket holds 10^4 rows, so below that scale Eq. (5) averages over a 10^4 -row region, the right answer to a different question. Section 6.4 measures how close $\hat{\rho}$ lands to the truth.

Classifying cells: what is bounded and what is not. A cell must be classified against the region as inside, outside or straddling its boundary, and the three outcomes are not symmetric: a cell called *outside* is dropped, so if that is ever wrong the query silently loses rows, while *inside* and *straddling* cells both produce a superset the exact predicate (Sect. 3) trims back. Only the outside decision therefore needs a bound rather than an estimate.

The bound used is

$$R_{\text{cell}}(k, p) = \min(\text{maxpixrad}(k), \max_i \angle(\mathbf{c}_{k,p}, \mathbf{v}_i)), \quad (6)$$

where $\mathbf{c}_{k,p}$ is the cell’s centre and $\mathbf{v}_i$ its four corners. The first term is the analytic worst case over the sphere used by the reference HEALPix implementation; the second is the cell’s own extent, which is smaller for most cells. A cell is outside when $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ for a cone of centre $\mathbf{q}$ and radius r , and correspondingly for each of a polygon’s bounding half-spaces. The margin $\epsilon = 2 \times 10^{-13}$ rad (40 pas) covers the accumulated floating-point error of the handful of dot products, cross products and inverse trigonometric calls involved, each carrying a few units in the last place; it is five orders of magnitude below the 0.4 mas resolution of an order-29 cell, so it costs nothing.

The inside test may be, and is, tighter: it uses the distances from the query centre to the cell’s own corners and edge midpoints directly. A tempting alternative for the outside test – distance to the chords between corners, inflated by the edge’s departure from its chord at the midpoint – is not a bound: near where an edge crosses its chord plane that departure collapses towards zero while the true one does not, by a factor reaching 64 at order 18, causing an error of $\sim 10^{-8}$ rad invisible to every test we had. Eq. (6) avoids it, at a cost of 2.5% more index ranges and no measurable time.

Regions as stored objects. A footprint is covered by a small number of cells written in the `NUNIQ` encoding of the MOC standard and stored one per row in an ordinary B-tree. The footprints containing a given position are those whose cell is one of that position’s ancestors, a short list of equality lookups (no duplicate elimination needed, since a MOC’s cells are disjoint by construction); refinement stops when cells become negligible next to the region, keeping the ancestor list short.

`skyregion` is one type internally tagged by kind rather than a family of per-shape types, so a single footprint column can hold circles and polygons together – an ObsCore `sregion` column, for instance, mixes the single-pointing circles of simple exposures with the polygons of non-trivial footprints in the same rows, covered by one function, `skycell_region_moc`, regardless of which kind a row holds. `pgSphere` has no equivalent: `scircle` and

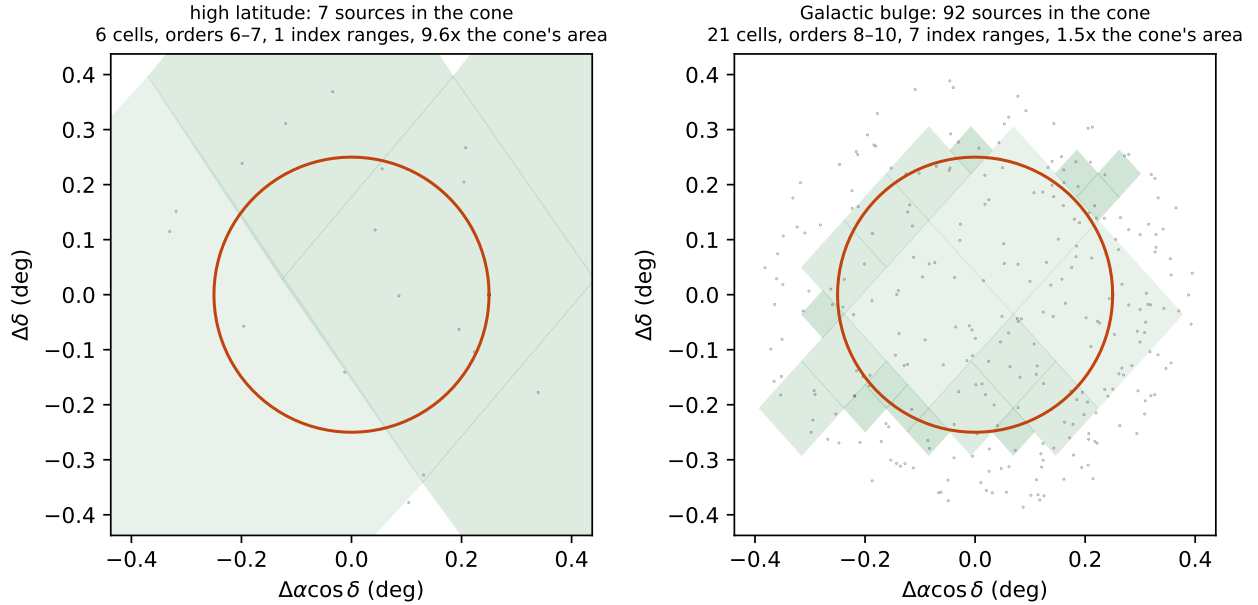


Figure 1: The same 0.25° cone covered in two places on the same catalogue. *Left:* at high latitude, 11 sources leave almost nothing to exclude, so the model buys one range of large cells (5.6 times the cone’s area). *Right:* in the bulge, 1183 sources make the false positives cost more than extra ranges save, so the model cuts two orders deeper (23 cells, 8 ranges, 1.3 times the area). Grey dots are catalogue sources, the heavy circle the query cone, the shaded quadrilaterals the covering cells.

`spoly` are separate types, so a mixed-shape column needs two columns, or a view over both, in its place.

2.3 The query surface

The extension exposes ADQL’s own vocabulary: types `skypos` and `skyregion` and functions `point`, `circle`, `box`, `polygon`, `contains`, `intersects`, `distance`, `area`, `coord1`, `coord2` and `centroid`, so that a translator emits the standard’s spelling rather than a dialect; the operator form `pos <@ circle(...)` is the one a planner can answer from an index, and the function form returns the same rows.

`skyregion`’s text representation is a second point of contact with the standard: it is IVOA STC-S (Rots 2007) – CIRCLE ICRS α δ r / POLYGON ICRS α_1 δ_1 ..., whitespace-separated – the same syntax an ObsCore `s_region` column already carries, so an `s_region skyregion` column casts directly from, and prints back as, the archive’s own STC-S text with no translation step; a stray flavor/refpos token some archives emit (TOPOCENTER) is accepted and discarded, and a non-ICRS frame is refused with an error rather than silently mis-read. This is independent of the query-time translation of ADQL’s CIRCLE(...)/POLYGON(...) inside a query (Table 1): STC-S is only the text form the stored value takes.

For hand-written SQL migrating from Q3C, `skycell_radial_query` and `skycell_join` take Q3C’s own argument lists as a prefix replacement,

both reducing to the canonical `skycell_cone`, which serves the cone search and the cross-match with one predicate since the only difference is whether the centre is a constant.

Archives also need the astrometry deciding *where* a source is at the epoch asked about. `skycell` implements the IVOA UDF registry’s names (`ivo_epoch_prop`, `ivo_epoch_prop_pos`, `ivo_apply_pm`, `ivo_healpix_index`, `ivo_healpix_center`) with the rigorous epoch propagation of the Hipparcos catalogue (ESA 1997), plus ICRS/Galactic/ecliptic frame conversions; a proper-motion-aware cone search is then an indexable cone widened by the largest motion of interest, intersected with an exact test on the propagated position. These are exercised by round-trip and published-value tests (epoch propagation returns Barnard’s star to sub-microarcsecond over a 16-year round trip) rather than benchmarked, so no performance claim is made for them.

Table 1 sets the three side by side for the constructs where they differ most. The right-hand column is mostly the identity, which is the point of adopting the standard’s spellings; the middle column is where the work goes, and two of its rows have no direct form at all – the remaining ADQL surface (`BOX`, `DISTANCE`, `COORD1/COORD2`, epoch propagation) translates the same way and is omitted for space.

Table 1: What a translator has to emit, for the ADQL constructs where the pgSphere-plus-Q3C pair and `skycell` differ most. Q3C supplies the fast cone and cross-match paths over scalar `ra/dec` but has no region type; pgSphere supplies region types but is slower at cones and cross-matches (Sects. 6.1, 6.3), so a translator targeting the pair must choose per query shape. `skycell` uses ADQL’s own spellings throughout.

ADQL	pgSphere + Q3C	skycell
POINT/CIRCLE/POLYGON('ICRS', ...)	<code>spoint(...), spoly('{(...),(...)}')</code>	<code>point/circle/polygon('ICRS', ...)</code>
BOX('ICRS', a, d, w, h)	<code>sbox</code> takes two corners, not a centre and extent – converted, and wrong at the poles, where a box is not a rectangle in (α, δ)	<code>box('ICRS', a, d, w, h)</code>
CONTAINS(POINT, CIRCLE) = 1	<code>q3c_radial_query(...)</code> or <code>pos <@ scircle(...)</code> – Q3C for speed, pgSphere if the position is stored as a type	<code>point(...) <@ circle(...)</code>
INTERSECTS(r_1 , r_2)	<code>r1 && r2</code> (pgSphere only; Q3C has no region type)	<code>r1 && r2 – intersects(...)</code> exists for ADQL compatibility but is not indexed
AREA/CENTROID(region)	<code>area(spoly)</code> returns steradians (scaled by $(180/\pi)^2$); no direct <code>CENTROID</code> for a polygon; no Q3C equivalent for either	<code>area(region), centroid(region)</code>
cross-match (CONTAINS against an uploaded table)	<code>q3c_join(...)</code> ; the pgSphere form is 2.4 times slower (Sect. 6.3)	<code>skycell_cone(...),</code> or <code>skycell_cone_ranges()</code> in a LATERAL

3 Implementation

The extension is written in C against the PostgreSQL 15+ server API: a geometry layer with no database dependencies (HEALPix arithmetic, region predicates, the covering) and a layer that connects it to the planner via a *planner support function* (`SupportRequestSimplify`, the mechanism PostGIS uses for `STDWithin`). Before path generation, a predicate such as `point('ICRS', ra, dec) <@ circle('ICRS',  $\alpha$ ,  $\delta$ ,  $r$ )` is replaced by a disjunction of range conditions on the key expression, conjoined with the exact test:

```
(cell BETWEEN lo1 AND hi1
OR cell BETWEEN lo2 AND hi2 OR ...)
AND skycell_in_region(point(ra, dec),
    circle(...), 0.29)
```

The support function builds this candidate expression from the predicate’s position argument and keeps it only if the relation carries an index on it, so the same query text works whether positions are two columns or a `skypos` column. The index conditions are then ordinary range predicates, so the planner estimates their selectivity from the same histogram, and the exact test’s last argument carries the covering’s occupied fraction, keeping the product of the two estimates close to the truth.

Two caches matter for planning cost: the density model and matching-index identity, per backend, dropped on statistics/relation changes; and coverings, memoised by region and statistics source, since a service plans the same cone many times. Measured on a TAP corpus, these are worth 45–70 and up to 50 μ s per plan respectively.

For joins, where the region is not a constant, the rewrite emits a fixed number of range slots per outer row (the structure `q3c_join` uses), while

`skycell_cone_ranges()` offers a variable number through a LATERAL join – a difference in plan shape that Sect. 6.3 shows matters.

Ingestion and maintenance. The index is an ordinary B-tree on an expression: an insert adds one 8-byte key, non-moving updates don’t touch it, and there is no method-specific rebalancing. Bulk ingestion benefits from loading in key order, the same order the queries exploit; and because the covering depends on the histogram, statistics matter more than for a GiST index – the area guard of Sect. 2.2 bounds the damage from a stale one, and `ANALYZE` on the 10-million-row corpora here takes 0.5–1.4 s.

4 What is proven, and what is tested

Proposition. Let R be a cone or convex polygon and let $\mathcal{C}$ be the covering computed for it. If (i) every point of a HEALPix cell at order k lies within $\text{maxpixrad}(k)$ of that cell’s centre, and (ii) every angle computed by the classification carries an absolute error below ϵ , then every order-29 cell that intersects R lies in one of the ranges of $\mathcal{C}$.

Proof sketch. The covering is produced by a descent that starts from cells whose union is the whole sphere and, at each step, either keeps a cell, drops it, or replaces it by its four children, whose union is the parent. The union of the kept cells is therefore reduced only where a cell is dropped, which happens only when it is classified outside. Taking $R_{\text{cell}} = \text{maxpixrad}(k)$ in Eq. (6), every point $\mathbf{x}$ of cell (k, p) satisfies $\angle(\mathbf{x}, \mathbf{c}_{k,p}) \leq R_{\text{cell}}$ by (i); hence if

$\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$, then by the triangle inequality on the sphere and (ii) every point of the cell is farther than r from $\mathbf{q}$, so the cell does not intersect the cone. The polygon case is the same argument applied to each bounding half-space. Truncation by the range cap, the area guard or the step budget only stops refinement, which keeps a cell whole, so it cannot remove a cell either. A cell containing a point of R intersects R and is therefore never dropped. $\square$

The proposition is conditional. Condition (i) is a property of HEALPix, not of this work: the reference implementation’s maxpixrad is analytic, verified numerically over 1.6×10^6 sampled boundary points at every order (largest observed ratio 0.9999). The tighter corner term in Eq. (6) additionally assumes the distance extremum over a cell is at a corner – supported by the same scan (ratio 0.96) but not proven; `skycell.exact_cells = off` rests the proposition on (i) alone, at the cost of Sect. 2.2. Condition (ii) is an assumption about floating-point arithmetic, not a theorem: ϵ is chosen five orders of magnitude above the plausible accumulated error and five below the leaf cell.

Because a covering that misses a cell silently loses rows, the implementation is also checked by exhaustion. The HEALPix layer is checked against known cell centres, round trips at all 30 orders, the nesting property of Eq. (1), equal area ($\chi^2/\nu = 1.14$ over 768 base cells), and the bounds of Sect. 2.2, over 1.6×10^6 sampled points including the poles and zone boundaries. The covering is checked against a 2-million-point clustered catalogue for cones from $1''$ to 120° and convex polygons; against 30 000 adversarial cones (poles, zone boundary, $\alpha = 0$, face edges, radii $0.02 \text{ mas} - 10^\circ$) with 200 points sampled inside each, half within a part per billion of the rim; and against 7266 adversarial polygons at the same places, with 866 537 points drawn inside them. All assert zero false negatives – this is the test that caught the error of Sect. 2.2. At the SQL level, two regression suites require every indexed answer to equal a sequential scan with the exact predicate: 240 cones, 60 polygons, both join forms and the MOC lookup in the first; the ADQL surface over 120 random regions, index-plan presence/absence, epoch-propagation and frame conversions in the second.

5 Benchmark

5.1 Three corpora

The *designed* corpus is 10 million synthetic sources with a density contrast resembling a real optical survey (35% uniform, 45% in a Galactic disk, 12% in a Gaussian bulge, 8% in 200 clusters), drawn deterministically from a seed – and invites the objection that its density structure was chosen by the authors of the method under test.

The *Gaia* corpus removes that objection as far as we can: source counts from the ESA Gaia archive for every order-9 HEALPix cell (3 145 728 cells of 0.11° , summing to the published DR3 total 1 811 709 771) give 10 million positions placed proportionally to each cell’s count, uniformly within it. It carries the real sky’s crowding – disk, bulge, Magellanic Clouds, scanning-law texture – down to the 0.11° map scale, below which it is locally Poisson by construction; no real positions are used, so it tests whether the cost model behaves the same on a density field the authors did not invent.

The *real* corpus removes the remaining objection: ten million real Gaia DR3 positions (`random_index` $< 10^7$, a precomputed random permutation, so a uniform thinning that preserves two-point structure at every scale, not only above a map cell), plus 1.6 million sources forming eight *complete* fields at full density (ω Cen, 47 Tuc, M4, M13, Baade’s Window, the Galactic centre, the LMC, and an off-cluster control) since thinning dilutes crowding to 0.55% of its true surface density. We use it for the density-estimator and ablation measurements, where sub-cell structure is the whole question, holding it twice – once in cell order, once in `random_index` order (random on the sky) – to measure physical layout as a variable rather than a fixed condition (Sect. 6.4).

5.2 Protocol

Q3C 2.0.5, pgSphere 1.5.2 and `skycell` each hold the same logical rows in their own natural layout, all three physically ordered along a space-filling curve so that heap locality is held equal (Table 2). Queries are the documented interface of each extension (`q3c_radial_query`, `spoint <@ scircle`, `skycell_cone`), and row counts are compared on every query.

The measurement is organised as *trials*: one query centre and radius, all three methods answering it back to back in an order drawn at random for that trial (and recorded, so an order effect can be measured rather than assumed away), repeated five times with the query set traversed in a random order each time. The analysis is paired – median over repetitions per method, ratio between methods for that same query, so the query’s own difficulty cancels – with a percentile bootstrap (4000 resamples) interval over queries; a ratio is called a difference only when its interval excludes 1.

Everything is measured *warm* (an untimed pass over the whole query set precedes the measurement, so no method sees a colder cache than another) and *cold* (server restarted, its 2 GB of shared buffers and the VM’s page cache empty, only the first repetition used). A separate randomized pass under `EXPLAIN (ANALYZE, BUFFERS)` records planning time, execution time, buffer pages and the planner’s row estimate apart from each other.

Table 2: The measured configuration.

Item	Value
Host	Apple silicon laptop, 11 cores, Docker Desktop Linux VM, 8 GB
Storage	NVMe SSD, through the VM’s file system
PostgreSQL	18.6; <code>shared_buffers</code> 2 GB, JIT off, <code>max_parallel_workers_per_gather</code> 0
Extensions	Q3C 2.0.5, pgSphere 1.5.2, skycell 0.4
<code>range_cost</code>	derived per relation (Eq. 7); 116 for the catalogues here
Rows	10^7 per corpus (Gaia corpus: 9 999 729); the Gaia corpus also at 5×10^7 (50 027 391)
Q3C	<code>cat.q3c(id, ra, dec, mag)</code> , B-tree on <code>q3c_ang2ipix(ra, dec)</code> , heap ordered by the same
pgSphere	<code>cat_sphere(id, ra, dec, mag, pospoint)</code> , GiST on <code>pos</code> , heap ordered by the HEALPix key
skycell	<code>cat_cell(id, ra, dec, mag, cellint8)</code> , B-tree on <code>cell</code> , statistics target 1000, heap ordered by <code>cell</code>
Maintenance	<code>VACUUM (FREEZE)</code> and <code>ANALYZE</code> on all three before measuring
Repetitions	5 (warm), 1 (cold); 1456 queries per repetition
Uncertainty	percentile bootstrap, 4000 resamples, over queries

Cone radii span $1''$ to 3° with 16–400 queries each, half the centres from catalogue positions (density-weighted) and half uniform on the sky. Cross-matches join a 200 000-row probe table (half catalogue sources displaced by $\sim 0.3''$, half uniform) to the catalogue; footprint queries ask which of 20 000 stored circles contain each of the 200 000 probes.

5.3 Reproducibility

Table 2 records the configuration in the detail needed to repeat or contest the measurement. Everything runs on one machine, with one core doing one query.

What this protocol still does not control. The corpora are synthetic, one designed by the authors and the other derived from a real density map but not from real positions (the real corpus, Sect. 6.3 and 6.4, is the exception). One machine, one storage device, one PostgreSQL configuration – absolute times are properties of that machine, only the comparisons are meant to travel – and the cold regime drops the page cache but cannot empty the device’s own. The three extensions are compared as configured here: a different Q3C query formulation or pgSphere index configuration could move their numbers, and Sect. 6.1 reports one such effect that dominates Q3C’s small-cone results.

6 Results

All three methods return identical row counts on every query of the grid. In the cross-matches pgSphere returns one or two more pairs out of $\sim 100\,000$, consistent with its 10^{-9} rad comparison tolerance counting sources exactly on the rim.

6.1 Cone searches

Table 3 and Fig. 2a are the primary result: paired ratios with intervals, on both corpora, warm and cold.

Warm, **skycell** is faster than pgSphere by 6–11% for radii from $1''$ to $1'$ and by 31–37% at 3° , on both corpora. Between $6'$ and 1° the two are, on the designed corpus, indistinguishable; on the Gaia corpus skycell is 3% *slower* at $6'$ (interval $[1.00, 1.08]$) and 8–15% faster at $30'$ and 1° . That measurement matters more than it sounds: measuring one method’s whole grid then the next instead gives ratios of 0.69–0.87 below $30'$ where the paired protocol finds 0.89–1.03, since the phase-ordered arrangement flatters whichever method runs later – here, **skycell**. The number above is the one we stand behind, and why every comparison here is paired inside a trial.

Cold, the picture is more favourable: **skycell** is faster at every radius on both corpora, by 5% at $1'$ and by a factor 2–2.5 at $30'$ and beyond, because it touches a third of pgSphere’s buffer pages at 1° (55 against 154) – a few microseconds of memory traffic when those pages are warm, the whole cost of the query when they are not.

Against Q3C, **skycell** is faster by a factor 4–50, but that number carries three different strengths. Against `q3c_radial_query` 2.0.5, which expands every call into 100 key ranges regardless of radius (about 300 buffer pages and 1.2 ms of planning before any row is read), the factor 4–50 is a comparison of *implementations*, and we stand behind it. Against `q3c_join`, which the Q3C authors intend for cross-matching and does not carry that fixed expansion, the two are level (Sect. 6.3) – the comparison informative about the two indexing *approaches*. About the cube quad-tree as a method we make no claim: a covering-aware radial query in Q3C would plausibly close most of the cone gap. pgSphere was used with its documented types and a GiST index at default settings, untuned further; no method was given prepared statements or plan reuse, which would help **skycell** most, since planning differs most between them. The order effect the randomization controls is small but real (going first costs pgSphere 19% and skycell –5% warm, Q3C flat), inflating the ratios’ variance rather than biasing them.

Does the advantage grow with the catalogue?

Ten million rows is 0.55% of Gaia DR3, and the argument that an index matters most when it does not fit in memory is an argument about scale, so

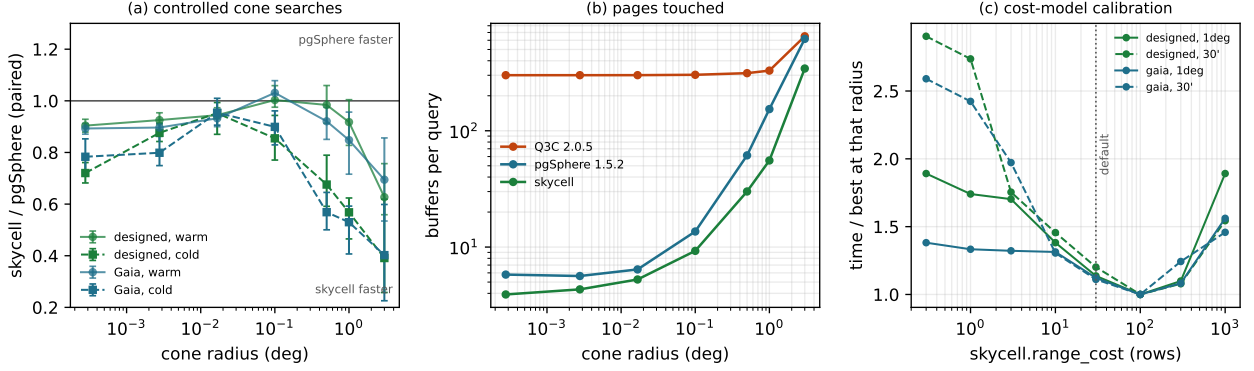


Figure 2: (a) Paired ratio of **skycell** to **pgSphere** against cone radius, 95% bootstrap intervals, both corpora and cache regimes; below the line skycell is faster. The warm advantage dips around 0.1° and returns at degree scales; cold it holds at every radius. (b) Buffer pages touched per query – what the cold curves are made of: at 1° skycell reads a third of **pgSphere**’s pages, and Q3C pays about 300 at any radius since 2.0.5 always expands a radial query into 100 key ranges. (c) The cost model’s one free parameter, swept over four decades and normalised to the best setting per radius; the curve is flat-bottomed over a decade either side of its minimum, why deriving it (Sect. 6.4) costs nothing in speed.

Table 3: Cone searches: paired ratio of **skycell** to each rival, with a 95% percentile-bootstrap interval over queries. A value below 1 means **skycell** is faster. **Bold** marks an interval that excludes 1 with skycell faster; *italic* marks one that excludes 1 with skycell slower; plain type means the two are not distinguishable. *n* is the number of distinct queries at that radius; each is answered five times per method (warm) or once (cold), all three methods in a randomized order within each trial.

radius	n	designed corpus		Gaia corpus					
		warm	cold	warm	cold				
<i>skycell</i> / <i>pgSphere</i>									
1''	400	0.90	[0.88, 0.93]	0.72	[0.68, 0.76]	0.89	[0.87, 0.92]	0.78	[0.72, 0.85]
10''	400	0.93	[0.91, 0.96]	0.88	[0.84, 0.92]	0.90	[0.88, 0.91]	0.80	[0.75, 0.86]
1'	300	0.94	[0.93, 0.97]	0.95	[0.87, 1.00]	0.93	[0.90, 0.96]	0.95	[0.90, 1.01]
6'	200	1.00	[0.98, 1.06]	0.85	[0.77, 0.94]	<i>1.03</i>	[1.00, 1.08]	0.90	[0.83, 0.96]
30'	100	0.98	[0.91, 1.06]	0.68	[0.59, 0.79]	0.92	[0.85, 0.98]	0.57	[0.50, 0.65]
1°	40	0.92	[0.83, 1.00]	0.57	[0.47, 0.62]	0.85	[0.72, 0.96]	0.53	[0.41, 0.59]
3°	16	0.63	[0.56, 0.76]	0.39	[0.26, 0.62]	0.69	[0.53, 0.86]	0.40	[0.22, 0.58]
<i>skycell</i> / <i>Q3C</i>									
1''	400			0.02		0.14		0.02	
1'	300			0.02		0.18		0.02	
30'	100			0.06		0.42		0.06	
1°	40			0.09		0.59		0.13	
3°	16			0.27		0.75		0.37	

we repeated the whole protocol on a 50-million-row Gaia-density corpus. There, pgSphere’s GiST index is 3442 MB against a 2 GB `shared_buffers` and no longer fits, while `skycell`’s 1072 MB B-tree still does.

The answer (Table 4) divides by region size. For 30’ and larger the advantage grows markedly: at 1° the warm ratio improves from 0.85 to 0.60 and the cold ratio from 0.53 to 0.31, because the buffer gap widens (152 against pgSphere’s 514, up from 55 against 154 at 10⁷ rows). For cones of an arcminute and below nothing improves – 1’’ stays at 0.89, 1’ moves from 0.93 to 1.02 – because both methods touch five to seven pages whatever the table size, so what remains is `skycell`’s planning (0.016–0.020 ms against pgSphere’s flat 0.009), which grows with source density as a denser sky makes Eq. (3) cut finer.

The rule that emerges is about pages rather than rows: `skycell` wins wherever a query has enough pages to save, and the larger the catalogue the more that is worth. An arcsecond cone on any catalogue is not the workload this index improves; a degree-scale selection, a polygon or a cross-match on a large one is.

6.2 Index size, build time, and what they cost at Gaia’s scale

Index size and build time are operational numbers – what an ingest, a restore or a schema migration has to pay – and the one result in this paper insensitive to protocol, so it is worth carrying to the scale an archive runs at (Table 5). The cost per row is flat between 10⁷ and 5 × 10⁷ rows (changing 0.5% for pgSphere, not at all for the two B-trees) – what the structures predict, an 8-byte key with its tuple header against a bounding volume per GiST entry – so extrapolating to Gaia DR3’s 1.81 × 10⁹ sources is a comparison of *deployments*: pgSphere answers cone searches and stores regions, `q3c_join` is what archives use for cross-matching (2.4 times faster than pgSphere at it, Sect. 6.3), and an archive wanting both carries both indexes today. `skycell` answers both from one, and that one is the smaller kind.

The memory column is the operational point: which cache regime a server is in is decided by whether the index fits beside the heap’s working set, and at DR3 scale a 64 GiB server holds `skycell`’s 38 GiB index with room to spare while the pgSphere-plus-Q3C pair needed for the same coverage wants 256 GiB. That factor-four gap is a procurement decision at these sizes, not a tuning one, and it lands exactly where our cold-cache measurements put `skycell` furthest ahead: on the 5 × 10⁷-row corpus, warm `skycell` against cold pgSphere (the comparison a 64 GiB archive actually faces, since only the former’s index is resident there) is one to two orders of magnitude apart at 30’ and beyond, and within

10% at small radii where both are memory-bound regardless.

This is an upper bound – a server whose index merely does not fit thrashes at some intermediate hit rate rather than missing every time, and the true figure depends on locality we have not measured at that scale – but the direction and order of magnitude hold: the difference between the two designs turns from percent to factors exactly when the index outgrows the machine, the regime a 10⁹-row catalogue puts a server in. Four further caveats: the extrapolation is linear and validated only over a factor five in row count; it covers the positional index alone, dwarfed in a real archive by the table and its other indexes; a partitioned deployment needs only its active partitions resident; and since the two B-trees are the same size, the real saving is not needing the geometry index at all, which rests on `skycell` answering the region queries pgSphere is kept for (Sects. 6.1, 6.3).

6.3 Cross-matching

Convex polygons of 0.05°–2° take a median 0.281 ms on the Gaia corpus against pgSphere’s 0.767 ms and Q3C’s 1.864 ms, and similarly on the designed corpus: a factor 2.7–4.4 either way, with pgSphere’s mean and 95th percentile far above its median where a polygon’s bounding volume is a poor fit.

Cross-matching a user’s uploaded target list against the catalogue is the operation the Gaia archive lists among its server-side capabilities (Salgado et al. 2017) and Q3C’s own speciality (`q3c_join`), so it deserves the cone searches’ protocol rather than a single end-to-end run: getting that protocol right mattered more here than anywhere else in this paper. At 50 million rows the catalogues and indexes are about 14 GB against 2 GB of `shared_buffers`, so whichever method runs first on a block of probes pays an I/O cost the others don’t (a first version of this measurement cost `skycell` a spurious factor ten by going first); each method is therefore warmed on its own query immediately before being timed, in a block-paired, randomized-order protocol, so what is compared is the work each does. At 50 million rows this gives `q3c_join` as the fastest established method by a factor 2.4 over pgSphere, and `skycell`’s `LATERAL` form level with it (0.78–0.92 of its time, not distinguishable) and 2.5–3.3 times faster than pgSphere; the fixed-slot join form is level with Q3C at 1’’ but 17% slower at 10’’, where a fixed slot count fits the covering less well.

Why level rather than better traces to the covering itself: the sky it scans, as a multiple of the sky asked for, peaks around 105 near 0.25–1’’ (where the cost model won’t cut finer, since the extra ranges would cost more than the area saved) and falls to 1.07 above 30°. A cross-match at 1’’ therefore does two orders of magnitude more area work than the region requires

Table 4: The same protocol at two catalogue sizes on the Gaia density field: paired ratio of **skycell** to pgSphere with 95% bootstrap intervals, and the planning, execution and buffer medians at 5×10^7 rows that account for them. At that size pgSphere’s index (3442 MB) exceeds **shared_buffers** while **skycell**’s (1072 MB) does not. **Bold**: interval excludes 1, **skycell** faster; *italic*: excludes 1, **skycell** slower.

radius	warm		cold		plan (ms)		exec (ms)		buffers	
	10^7	5×10^7	10^7	5×10^7	pgS	sky	pgS	sky	pgS	sky
1''	0.89	0.89	0.78	0.71	0.009	0.016	0.025	0.014	6	5
10''	0.90	0.94	0.80	0.88	0.008	0.016	0.025	0.014	5	5
1'	0.93	1.02	0.95	0.95	0.009	0.020	0.033	0.023	7	6
6'	<i>1.03</i>	1.00	0.90	0.89	0.009	0.031	0.059	0.049	14	19
30'	0.92	0.72	0.57	0.49	0.009	0.075	0.550	0.332	141	74
1°	0.85	0.60	0.53	0.31	0.009	0.087	1.976	1.040	514	152
3°	0.69	0.58	0.40	0.23	0.014	0.143	10.686	5.801	1080	676

Table 5: Index cost at 10 million rows and extrapolated to Gaia DR3 scale (1.81×10^9 sources, linear from the 5×10^7 measurement, optimistic for all three equally; per-row cost is flat to 0.5% between the two measured scales). The last column is the smallest conventional server memory holding the index resident with room for the heap’s hot pages.

index	10 million rows			at 1.81×10^9 (Gaia DR3)		
	MB	build (s)	ANALYZE (s)	GiB	build	mem GiB
Q3C	214	2.2	0.23	38	13 min	64
pgSphere	685	42.3	0.46	122	2.7 h	192
skycell	214	1.4	0.57	38	14 min	64
pgSphere + Q3C	—	—	—	160	2.9 h	256

Table 6: Cross-matching at the radii the operation is used at, on **real Gaia DR3 positions**: geometric mean of the paired per-cell ratio over three upload sizes (10^3 , 10^4 , 10^5) and two target distributions, method order drawn per cell, each method warmed on its own query, three repetitions. Below 1 means **skycell** is faster.

radius	vs q3c_join		vs pgSphere	
	LATERAL	slots	LATERAL	slots
0.5''	0.96	1.26	0.35	0.46
1''	0.80	1.12	0.32	0.45
1.5''	1.12	1.66	0.28	0.42
5''	1.08	1.29	0.35	0.42
all	0.98	1.32	0.33	0.44

and still draws level with **q3c_join**; the same index at degree scale, where the covering is within a third of the region, wins outright.

We repeated this on the real corpus at the radii cross-matching actually uses – optical work at 1–1.5'', radio up to 5'' (Table 6; an earlier 30'' point was an association radius, not a cross-match one, and is dropped). Over that range the **LATERAL** form is level with **q3c_join** (geometric mean 0.98 over twenty-four cells) and 3.0 times faster than pgSphere; the fixed-slot form is the weaker of our two here (1.32 of **q3c_join**) and degrades further above these radii, a bound on that form rather than a general weakness. The claim this supports is narrow: on the operation Q3C is built for, **skycell** is Q3C’s equal rather than its better, while being 2.5–3.3 times faster than pgSphere, with the same index that answers the region queries of Sect. 6.1.

For stored footprints, the MOC-in-a-B-tree ar-

rangement answers the same 200 474 point-in-region pairs in 0.76 s against pgSphere’s 0.97 s, with an index that is larger (9.5 MB against 1.1 MB) and slower to build (0.9 s against 0.03 s), since a region becomes several rows rather than one – measured one method after another rather than in randomized trials, since each is a single query of seconds; the ordering effect of Sect. 6.1 applies.

Estimates, plans and joins. A spatial predicate is rarely alone. We ran three shapes – a cone with a catalogue cut, the same joined to an observation table, and a three-way join – through each method, recording the estimate, the plan strategy and the time separately: **skycell**’s estimate advantage carries over (median error 1.15–1.88 at 30'–1° against 1.63–2.33 for Q3C and 2.50–3.00 for pgSphere), but the plans are the same. Across 135 queries at 1'–3° all three methods chose the same strategy for every query except one at 3°, where **skycell**’s estimate produced a hash join 1.29 times faster than pgSphere’s nested loop; execution times are otherwise within noise. The honest summary: better estimates are demonstrated, better plans and better join times are not – an accurate estimate is insurance against a query we did not write, and we have not shown that insurance paying off on one we did.

6.4 The cost model: its parameter, its input, and what it is worth

Equation (2) has one free parameter, c_{range} , and it need not be a parameter at all: it is a ratio of two costs PostgreSQL already models, one more index

Table 7: The density estimate against the truth, on **real Gaia DR3 positions**. $\hat{\rho}$ is what the cost model reads; ρ is counted from the same relation; both in rows sr^{-1} . Six positions spanning four decades of surface density, radii 0.01° – 1° .

field	median $\hat{\rho}/\rho$	range
ω Cen	0.08	0.05–0.51
47 Tuc	0.12	0.06–0.93
LMC	0.76	0.25–0.93
Baade’s Window	0.80	0.68–1.01
N galactic pole	0.90	0.53–1.28
Galactic centre	3.74	2.53–3.95

range (a B-tree descent, $(\lceil \log_2 N \rceil + H + 1) \cdot 50 c_{\text{op}}$) against one false-positive row (its index entry, heap tuple, the exact predicate, and its heap page amortised over the rows sharing it):

$$c_{\text{range}} = \frac{(\lceil \log_2 N \rceil + H + 1) 50 c_{\text{op}}}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + c_{\text{page}}/n_{\text{pp}}}, \quad (7)$$

with $n_{\text{pp}} = \text{reltuples}/\text{relpages}$. Every input is already at hand, so `skycell.range_cost` defaults to evaluating it, per-relation rather than per-installation (116 on the catalogues at 120 rows/page; 93 on a wider-row ObsCore relation at 54 rows/page). Placed in the same trials as the fixed constant it replaces, the two are indistinguishable, because the cost curve is flat (sweeping over four decades moves the median time under 25% between 3 and 300): what the derivation buys is not speed but that the parameter stops being a number someone has to choose. The order it picks is likewise close to optimal (about 3% against one order finer at $6'$), for the same reason – the model does not need to be right, only to land in the flat region.

The model’s input, $\hat{\rho}$ from Eq. (5), was called unbiased in an earlier version of this work – true of positions resampled from a density map with no structure below 0.11° , but not of real Gaia DR3 positions. Table 7 compares $\hat{\rho}$ with ρ counted from the same relation: good to about $\pm 25\%$ on ordinary sky, *low by a factor 8–12* in a globular cluster core, *high by a factor 3.7* at the Galactic centre, as the histogram resolves a cluster while a query sits inside it and averages it away once the query is larger (the failure Sect. 2.2 predicts, now measured on real positions rather than argued about). The consequence is smaller than the error suggests: since $s^* \propto \sqrt{\rho}$, an order of magnitude in ρ is a factor three in cell size, and the cost curve is flat across that – the covering chosen in a cluster core, where $\hat{\rho}$ is wrong by an order of magnitude, is within 1.09–1.43 of the best fixed order, no worse than where the estimate is accurate. The estimator should be judged by the decision it produces, not the number it reports.

The comparison with pgSphere confounds three things, which separate as follows on the real Gaia corpus. *Heap clustering* is a large absolute effect and almost no differential one: unclustered rows cost 1.38–

2.08 times more for `skycell` and 1.55–2.22 times more for pgSphere across $10''$ – 1° , but the ratio between them barely moves (0.64 clustered against 0.60 unclustered at 1°), so physically ordered tables do not account for the comparison. *Adaptive resolution* is worth a great deal: against every fixed order 4–13, the penalty for choosing badly is 243–830 $\times$, and the best fixed order moves from 12–13 at $10''$ to 7–8 at 0.5° , while the cost model lands within 1.02–1.58 of the best without knowing the answer. *The index representation* is isolated by comparing against Q3C instead, since both are B-trees over an `int8` key and only the covering strategy differs (Sect. 6.1, with the caveat of Sect. 6.1).

7 Inside a TAP service, and where the crossover is

The service-level A/B. A benchmark of index operations is not a benchmark of an archive. We ran the same build inside `egernia` (Delli Veneri & Salgado 2026), a TAP 1.1 server, against its published 500 096-row ObsCore corpus: one database holding both indexes, two otherwise identical service processes differing only in how they translate CONTAINS, interleaved rung by rung over six query classes at 1, 8 and 32 concurrent clients, three repetitions each. Both targets pass the conformance gate and the cross-server agreement gate on all eleven portable classes, with identical rows through both translations. The performance result is a tie – all 18 comparison cells fall inside the tie rule (throughput ratios 0.96–1.06) – although at the database level `skycell` is 1.2–1.45 times *slower* per query on that corpus. Both facts share a cause: a cone over half a million rows costs a fraction of a millisecond either way against roughly 10 ms of CPU in the service’s own Python, so the database is 1–3% of a request and a 0.08 ms difference in it cannot move throughput.

Where the crossover is. That explanation makes a testable prediction. We built the same ObsCore-shaped relation – observations in groups of 128 products sharing a field centre, clustered by collection and field, which is what makes a GiST bitmap scan so effective on this data – at three sizes, and ran the same four cone classes against both indexes (Table 8). The crossover is real but not a single number: for the cone with catalogue cuts (Q07) and the degree-scale cone (Q06), `skycell` is ahead at every size and pulls further ahead as the relation grows (a seventh of pgSphere’s buffer pages at Q06, a quarter at Q07); the empty cone (Q12) is level. The small-cone class (Q05, 0.15°) we cannot pin down – 2.16 times slower immediately after the build, level (0.94) once thoroughly warm, and not reproducible on this host to better than a factor of two – but the deficit

Table 8: Crossover on an ObsCore-shaped relation, measured in randomized paired trials (30 centres, 3 repetitions, each method warmed on its own query first): total query time as the ratio `skycell/pgSphere`, with buffer pages at the largest size. Q05: 0.15° cone; Q06: 2° cone; Q07: 0.5° cone with calibration and time cuts; Q12: an empty $0.7''$ cone. The Q05 row is regime-dependent and should not be read as a single number (text).

class	skycell / pgSphere			buffers at 10^7	
	5×10^5	2×10^6	10^7	pgSphere	skycell
Q05	1.84	1.81	2.16	282	152
Q06	0.92	0.87	0.67	6052	830
Q07	0.84	0.76	0.71	712	182
Q12	1.13	1.07	1.00	5	6

is at least localised, in execution (0.324ms against 0.161) rather than planning or the exact predicate.

When the index no longer fits in memory.

Section 6.2 argues from index size alone that a host must reach a point where `pgSphere`’s index no longer stays in memory while `skycell`’s still does; we tested the arithmetic directly. In a 3 GiB container (768 MB shared buffers) holding a 22 GB, 20.5-million-row ObsCore-shaped relation – GiST index 1162 MB ($1.51\times$ the buffer pool), `skycell` 439 MB ($0.57\times$) – a stream of 3500 uniform 2° cones (covering the sphere about once, needed to make the cache matter at all) was run against each index in its own phase, server restarted between phases, both phase orderings tried. `skycell` answers 4.2–4.8 times faster, serving 53% of its index accesses from the pool against `pgSphere`’s 9%. Two qualifications: the 22 GB heap alone takes 705–734 MB of the pool in every phase, so *neither* index is actually resident – what is measured is a smaller index winning a contested pool, not a clean fits/doesn’t-fit threshold – and running both translations in a single interleaved pass instead (the design we tried first, where each leaves the shared heap pages warm for the other) puts the gap at $1.6\times$ rather than 4.2.

8 Discussion

The advantage is not universal, and stating where it stops is as informative as stating where it holds. Warm, between $6'$ and 1° , `skycell` is indistinguishable from `pgSphere` on the designed corpus; on the Gaia corpus it is 3% slower at $6'$ but 8–15% faster at $30'-1^\circ$. On a 500 000-row ObsCore relation it is slower; on the small-cone ObsCore class it is slower in one cache regime and level in another, and we do not claim that row. Better row estimates did not, on the join shapes we wrote, produce better plans. And every number here comes from one machine and one storage device, with a density field standing in

for real catalogue positions except where Sects. 6.3 and 6.4 say otherwise.

What remains expensive is planning: a covering costs 0.015–0.12ms even cached against 0.006–0.008ms for a GiST predicate that needs no such step, the whole of `skycell`’s disadvantage at $6'$. We prototyped an SP-GiST operator class over the cell hierarchy to remove this architecturally rather than amortise it, and found it is not the clean fix it looks like on paper: the compression needed to keep the tree compact costs correctness margin elsewhere, shrinking the duplicate-position tolerance by roughly an order of magnitude. That prototype is exploratory and not part of the released extension, so we report only the qualitative conclusion – an index descent that avoids planning ranges remains open, not solved.

The density synopsis is the other limit: an equi-depth histogram cannot see a cluster smaller than one of its buckets, and raising the statistics target does not help, since a denser region gets narrower buckets and the interpolation stays scale invariant. The area guard bounds the damage; a custom `typanalyze` storing a multi-order count map would estimate what the covering actually needs, and we have implemented one as an option, but have no measurement showing it helps.

9 Conclusions

`skycell` keeps Q3C’s compact integer key and B-tree while answering the region queries `pgSphere` was built for, by treating the depth of a covering as a cost question whose answer follows from the local source density and the price of an index range. The result is a single index that holds its own against each established extension on the workload the other one owns. Against `pgSphere` it is faster for cone searches at most radii warm and at every radius cold, by a factor of up to 2.5, and answers convex polygons 2.7–4.4 times faster. Against Q3C, at the cross-match Q3C is built for, it is Q3C’s equal and 2.5–3.3 times faster than `pgSphere` at the same operation. It gets there with an index a third the size of `pgSphere`’s – 38 GiB against 160 GiB at Gaia DR3 scale for the `pgSphere-plus-Q3C` pair that covers the same ground, the difference between a 64 GiB server and a 256 GiB one – built thirty times faster, with row estimates twice as accurate, and a cost model whose one parameter is derived from the relation’s own statistics rather than configured.

`skycell` also closes two gaps that neither established extension closes alone. It speaks ADQL’s own vocabulary as directly as `pgSphere`, with an operator form a planner can answer from an index; and its stored regions read and write as IVOA STC-S text itself, so an ObsCore `s.region` column round-trips against an archive’s own text with no trans-

lation step. Because `skyregion` is one type tagged by kind rather than a family of per-shape types, a single column holds circles and polygons together – exactly the case, a mixed-shape footprint column, that `pgSphere` cannot represent in one column at all. Combined with a single compact index that serves both point and region queries, this makes `skycell` a strong candidate for the geometry layer of an IVOA TAP service, where ADQL compliance, STC-S storage and mixed-shape footprints are what an archive already needs to provide, and where Sect. 7 shows it dropping into a working TAP server without changing the service’s own behaviour.

The advantage is a property of the workload rather than a universal one, and scale sharpens it rather than erasing it: raising the catalogue to 50 million rows widens the lead for regions of 30′ and above – to a factor 3.2 at 1° cold – while cones of an arcminute and below stay level with `pgSphere`, because both methods touch a handful of pages there at any table size and what is left is planning. That is also where a small relation, or a TAP server whose own cost is dominated by Python, favours `pgSphere` or leaves the choice immaterial. Choosing an index carefully matters when the scan dominates; we have tried to say precisely which queries those are.

Data availability

The extension, its test suites, the benchmark harness and the raw results of every measurement quoted here are available at <https://github.com/jesusjuansalgado/skycell> under the MIT licence. Both corpora are generated from seeds by the scripts recorded there, reproducible without transferring data; `bench/run.sh` and scripts 07–10 reproduce every table, and the TAP A/B of Sect. 7 is recorded there too.

Acknowledgements. This work has made use of data from the European Space Agency (ESA) mission *Gaia* (<https://www.cosmos.esa.int/gaia>), processed by the *Gaia* Data Processing and Analysis Consortium (DPAC, <https://www.cosmos.esa.int/web/gaia/dpac/consortium>), funded by national institutions participating in the *Gaia* Multilateral Agreement. Only per-cell source counts were used, to build the density field of Sect. 5.1.

The implementation, benchmark harness and analysis were developed with the assistance of Anthropic’s Claude Code. Because that bears on how the results should be read: the geometric predicates and covering are checked by the brute-force suites of Sect. 4 against independent computations over millions of adversarially placed points rather than expected outputs; every indexed answer in the regression suites must equal a sequential scan with the exact predicate; and the error of Sect. 2.2 was found by those tests rather than by inspection, the argument for relying on them. The benchmark harness was itself audited by reproducing its headline comparison under a

different protocol (Sect. 6.1), which is how the difference between the two came to light.

References

- Chilingarian, I., Bartunov, O., Richter, J., & Sigaev, T. 2004, in ASP Conf. Ser. 314, *Astronomical Data Analysis Software and Systems XIII*, ed. F. Ochsenbein, M. G. Allen, & D. Egret (San Francisco: ASP), 225
- Delli Veneri, M., & Salgado, J. 2026, *egernia*: an IVOA TAP 1.1 server for the SKA Observatory, Zenodo, doi:10.5281/zenodo.22845690
- Demleitner, M., Neves, M. C., Rothmaier, F., & Wambsganss, J. 2014, *Astron. Comput.*, 7–8, 27
- Dowler, P., Rixon, G., Tody, D., & Demleitner, M. 2019, *IVOA Recommendation: Table Access Protocol Version 1.1*
- ESA 1997, *The Hipparcos and Tycho Catalogues*, ESA SP-1200
- Fernique, P., Boch, T., Donaldson, T., et al. 2014, *IVOA Recommendation: MOC – HEALPix Multi-Order Coverage map Version 1.0*
- Gaia Collaboration, Prusti, T., de Bruijne, J. H. J., et al. 2016, *A&A*, 595, A1
- Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, *ApJ*, 622, 759
- Gray, J., Nieto-Santisteban, M. A., & Szalay, A. S. 2006, *The Zones Algorithm for Finding Points-Near-a-Point or Cross-Matching Spatial Datasets*, Microsoft Research Technical Report MSR-TR-2006-52
- Koposov, S., & Bartunov, O. 2006, in ASP Conf. Ser. 351, *Astronomical Data Analysis Software and Systems XV*, ed. C. Gabriel, C. Arviset, D. Ponz, & S. Enrique (San Francisco: ASP), 735
- Kunszt, P. Z., Szalay, A. S., & Thakar, A. R. 2001, in *Mining the Sky*, ed. A. J. Banday, S. Zaroubi, & M. Bartelmann (Berlin: Springer), 631
- Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. 2001, *IEEE Trans. Knowl. Data Eng.*, 13, 124
- Ortiz, I., Lusted, J., Dowler, P., et al. 2008, *IVOA Recommendation: Astronomical Data Query Language Version 2.0*
- Rots, A. H. 2007, *IVOA Recommendation: Space-Time Coordinate Metadata for the Virtual Observatory Version 1.33*
- Salgado, J., González-Núñez, J., Gutiérrez-Sánchez, R., et al. 2017, *Astron. Comput.*, 21, 22

Salgado, J., et al. 2020, in ASP Conf. Ser. 522, Astronomical Data Analysis Software and Systems XXVII, ed. P. Ballester, J. Ibsen, M. Solar, & K. Shortridge (San Francisco: ASP), 307